

Matrix Multiplication Performance Analysis

Single-core behavior, cache effects, blocking, and OpenMP scalability

Group g16

Filipe Camacho - up202208040

Miguel Neri - up202006475

Afonso Bouça - up202304970

Objectives and experimental setup

What we measured and how the experiments were organized.

Algorithms

- OnMult (baseline)
- OnMultLine (sequential access)
- OnMultBlock (block sizes 128, 256, 512)

Languages and sizes

- C++ compiled with -O2 and Java versions
- 1024 x 1024 to 3072 x 3072 for cross-language comparison
- C++ extended up to 10240 x 10240 for large-matrix tests

Metrics

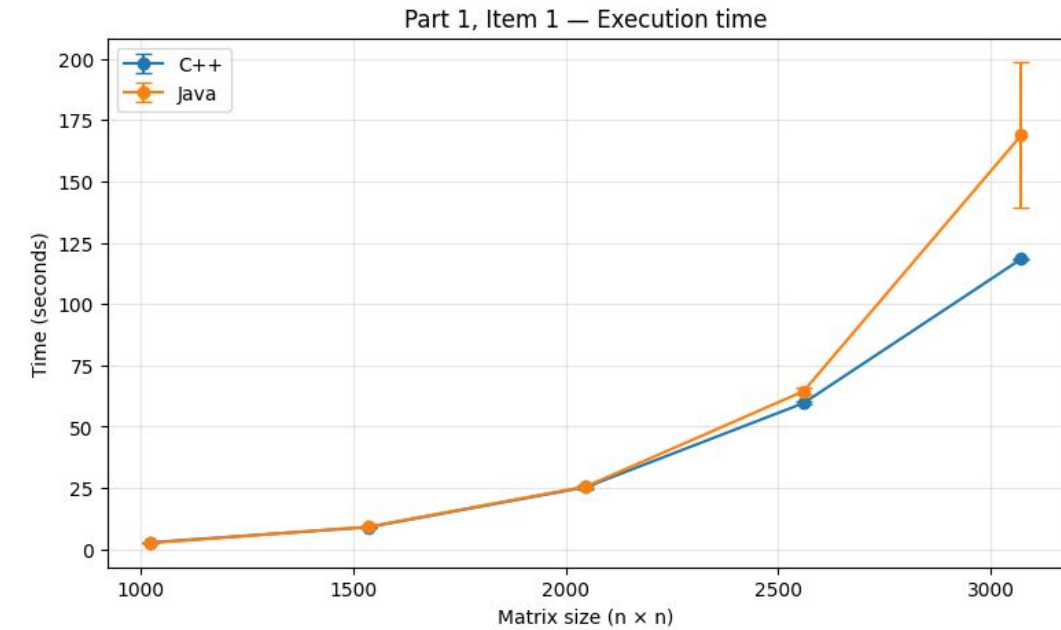
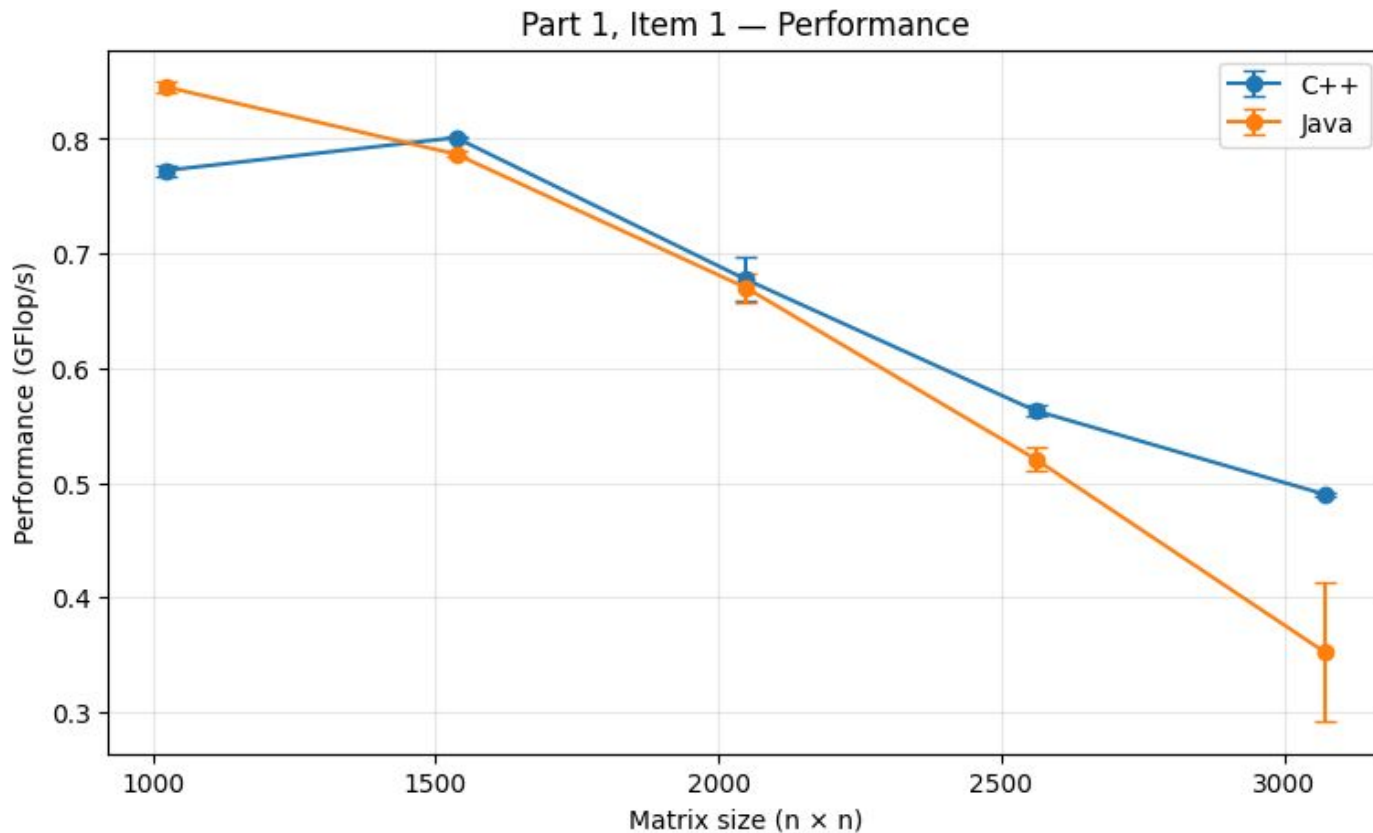
- Execution time and GFLOP/s
- L1 and LLC miss rates, IPC, task clock
- Speedup and efficiency for OpenMP experiments

Method

- Each configuration executed 5 times
- Results aggregated as mean with variability/error bars
- OpenMP tested with 4-thread mappings and with 4-24 threads at 8192 x 8192

Baseline OnMult: C++ vs Java

The baseline multiplies one row of A by one column of B, which gives poor spatial locality for matrix B.

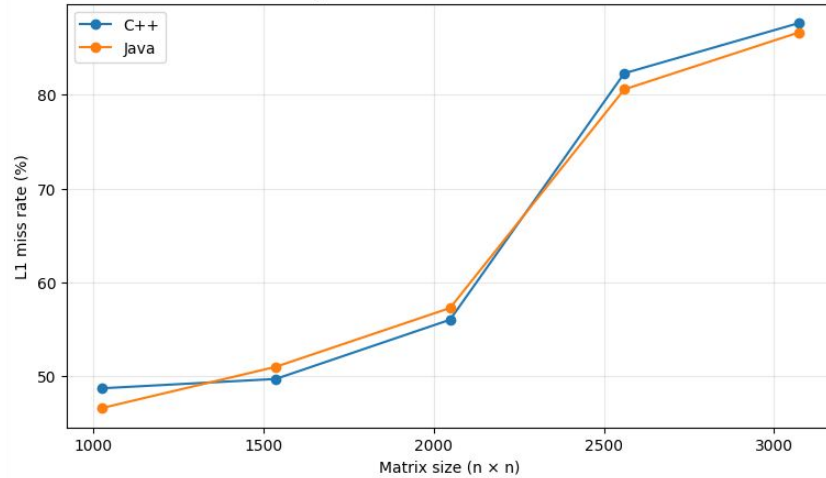


- Both baseline versions stay below 0.85 GFLOP/s.
- Java is slightly better at smaller sizes, but C++ becomes clearly better as N grows.
- At 3072 x 3072, C++ reaches about 0.49 GFLOP/s in 118 s; Java reaches about 0.35 GFLOP/s in 169 s.
- The baseline already suggests that memory access, not arithmetic throughput, is the limiting factor.

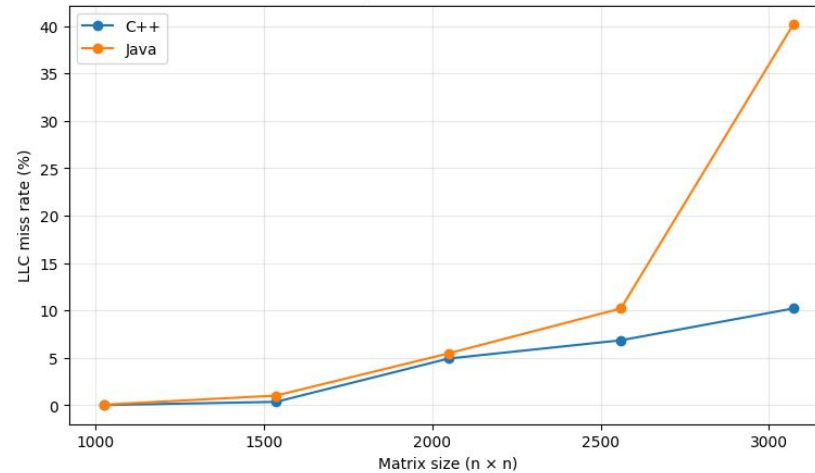
Why the baseline stalls

Cache miss rates increase with matrix size and IPC drops, which is consistent with a memory-bound execution.

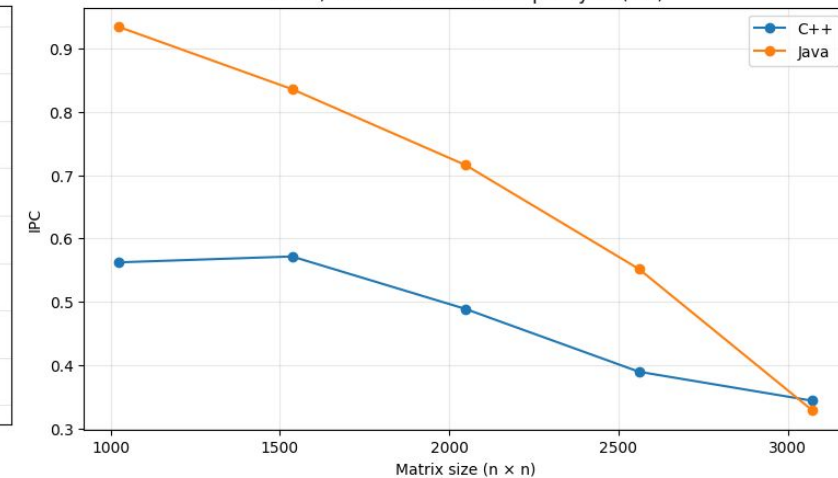
Part 1, Item 1 — L1 data-cache miss rate



Part 1, Item 1 — LLC miss rate



Part 1, Item 1 — Instructions per cycle (IPC)

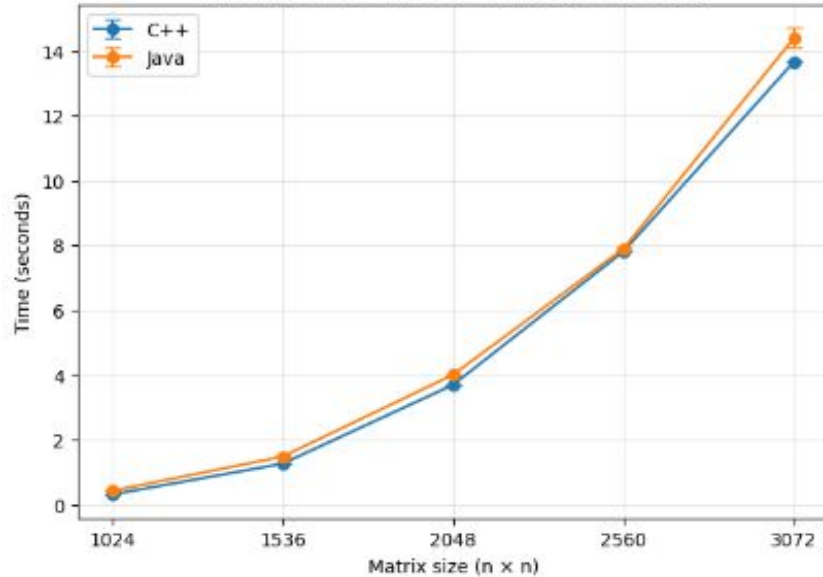


- L1 miss rate rises sharply for both languages as matrices outgrow the cache-friendly region.
- Java shows a much stronger LLC miss-rate increase and reaches about 40% at 3072, versus about 10% in C++.
- IPC falls from roughly 0.56 / 0.93 at 1024 to around 0.34 for both implementations at 3072, showing that the CPU spends more time waiting on memory.

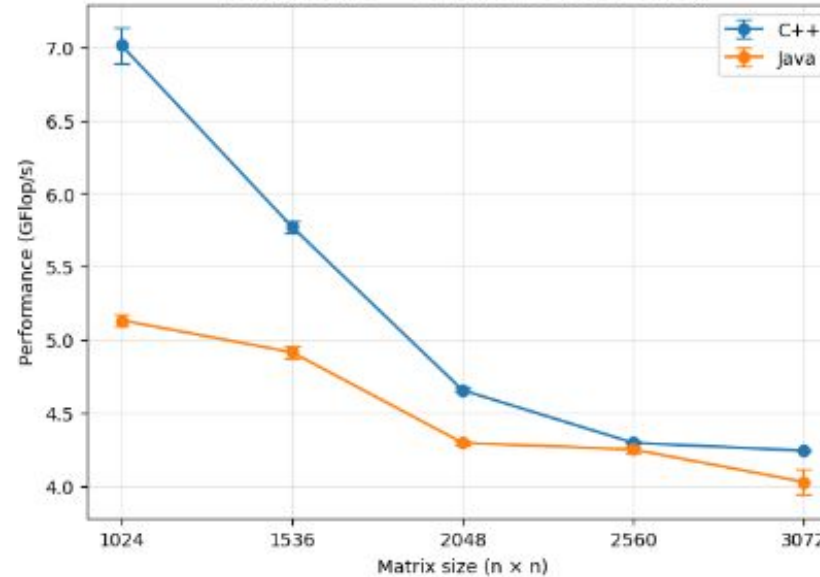
OnMultLine: same arithmetic, much better locality

Reordering the inner loops makes matrix B much more sequential in memory.

Part 1, Item 2 — Execution Time (1024-3072)

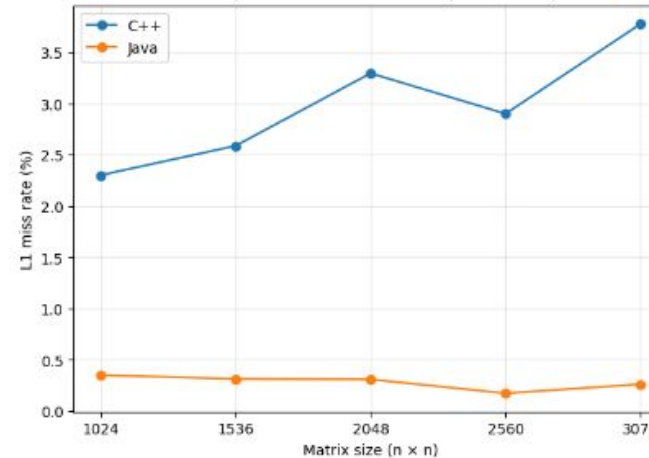


Part 1, Item 2 — Performance (1024-3072)

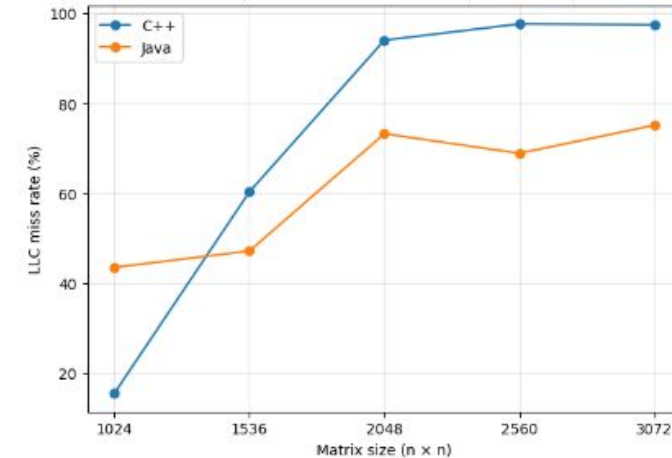


L1 miss rate falls dramatically:
about 2-4% in C++ and below
0.4% in Java.

Part 1, Item 2 — L1 Miss Rate (1024-3072)



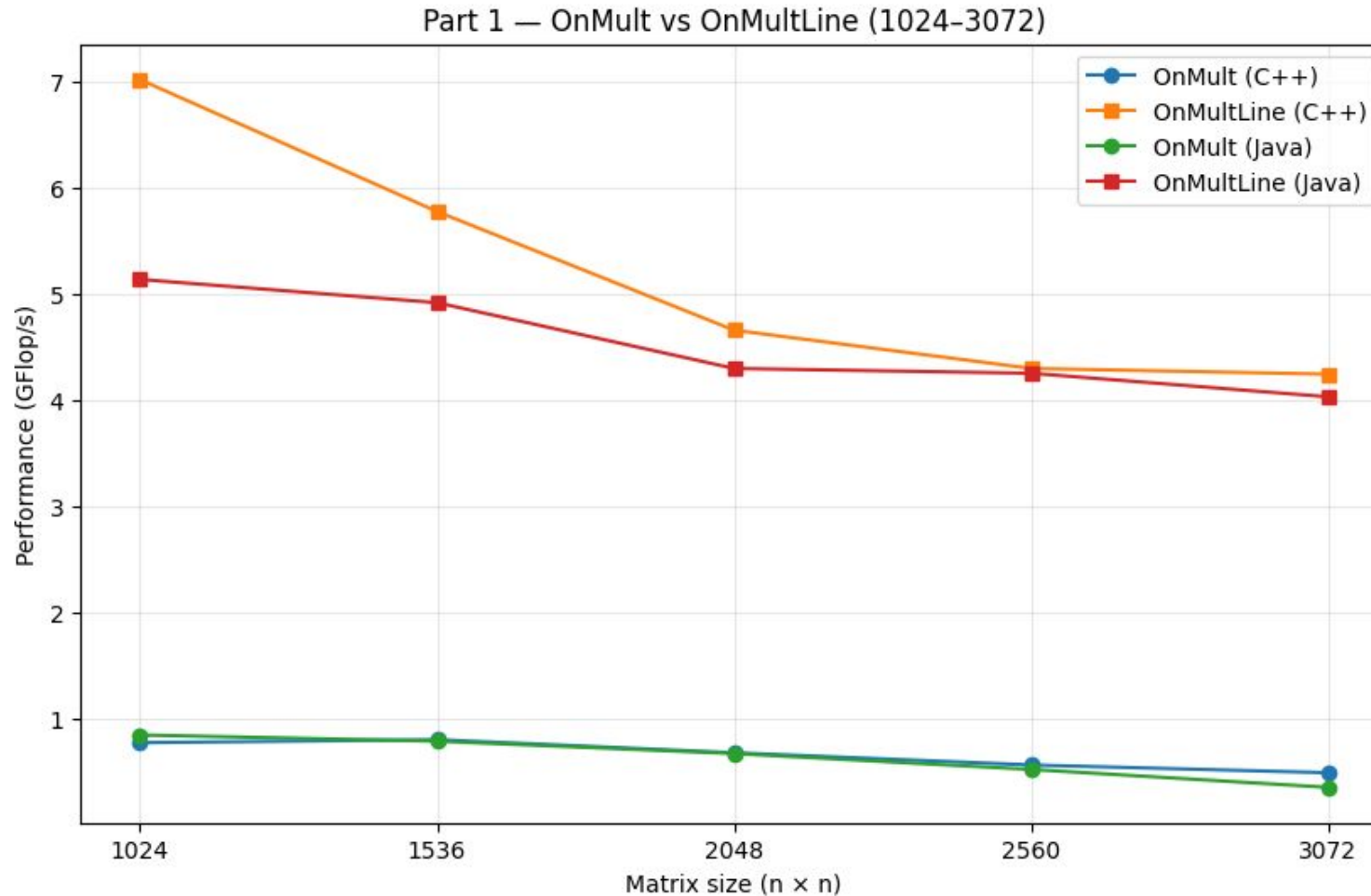
Part 1, Item 2 — LLC Miss Rate (1024-3072)



C++ now reaches about 7.0 GFLOP/s at 1024 and stays above 4.2 GFLOP/s at 3072.

Direct comparison: OnMult vs OnMultLine

The arithmetic is the same; the major difference is the order in which memory is touched.



C++ gain

0.77 -> 7.0 GFlop/s at 1024 (~9x)
0.49 -> 4.24 GFlop/s at 3072 (~8.7x)

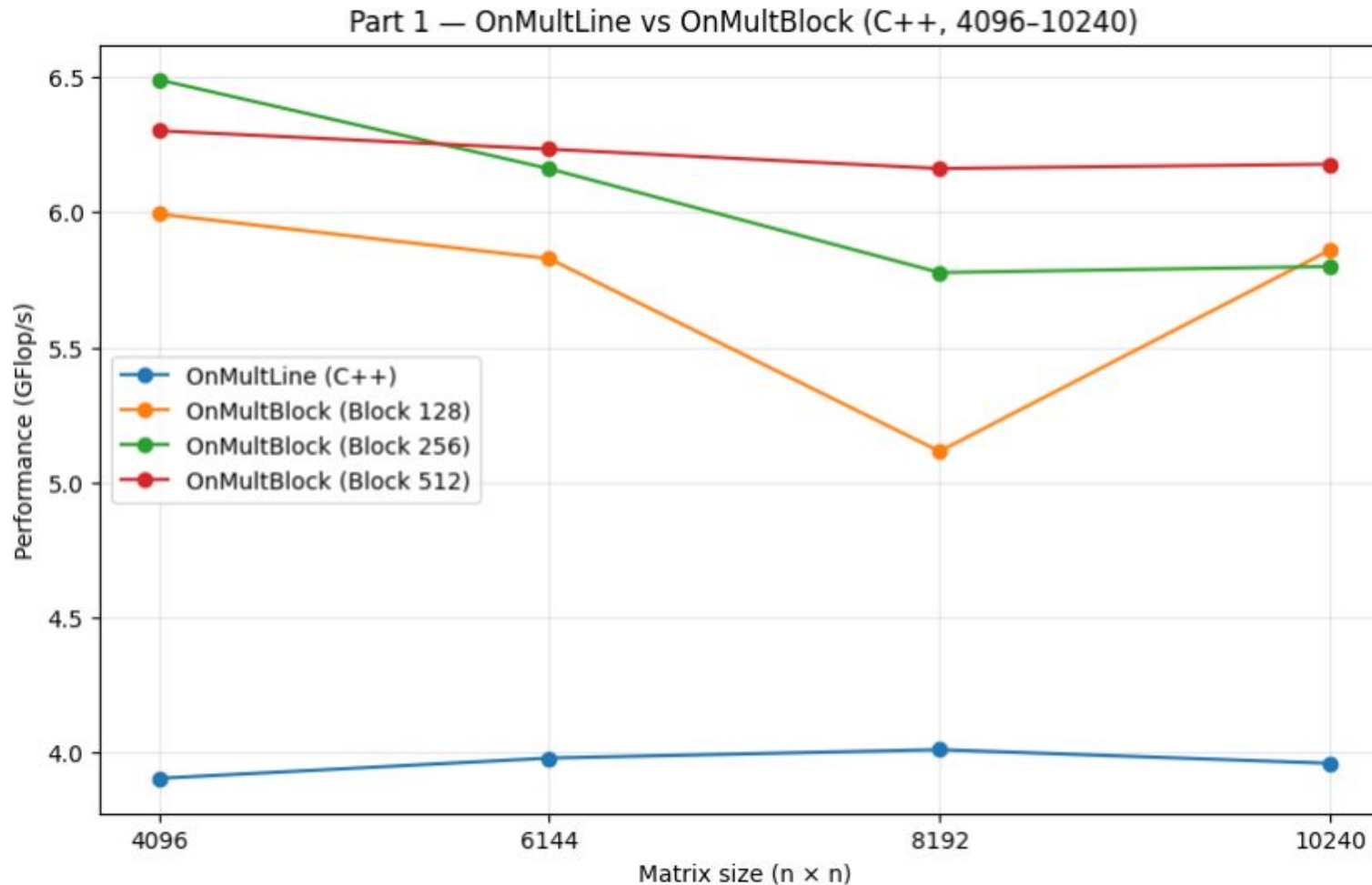
Java gain

0.85 -> 5.13 GFlop/s at 1024 (~6x)
0.35 -> 4.02 GFlop/s at 3072 (~11x)

- The performance jump comes from better use of cache lines and improved spatial locality.
- This is the clearest single-core result of the project: loop ordering alone changes performance by almost an order of magnitude.

OnMultBlock for large matrices

Blocking improves reuse by working on smaller tiles that fit more naturally in the cache hierarchy.

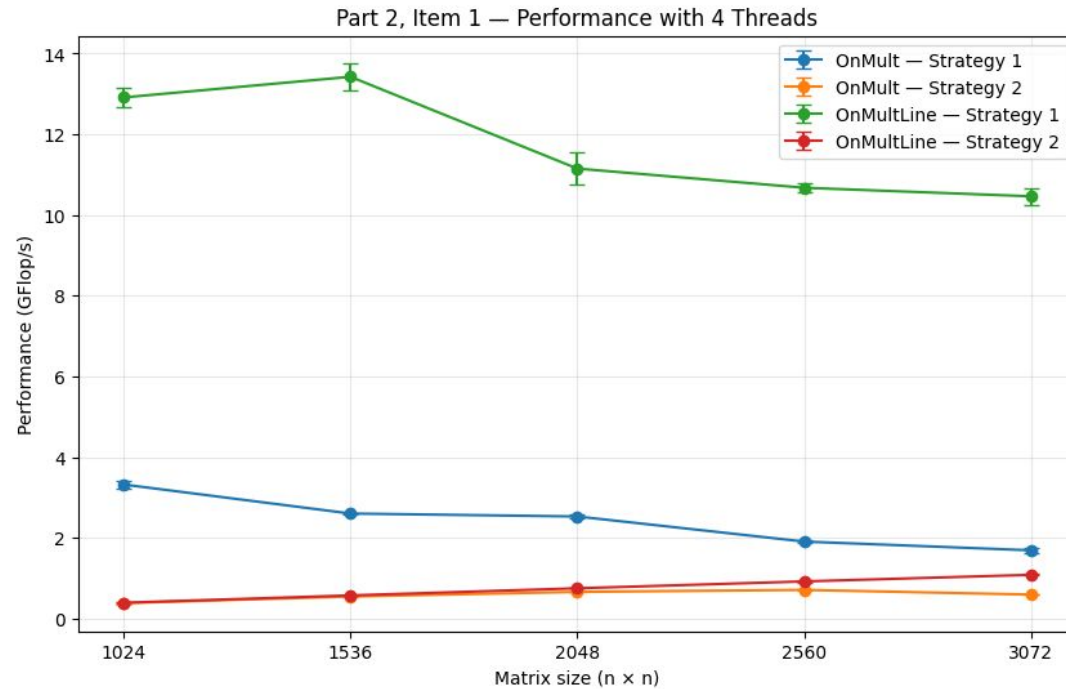


- Blocking overtakes plain OnMultLine once the matrices are large enough to stress the cache hierarchy.
- Block 256 is best at 4096, reaching about 6.49 GFLOP/s.
- Block 512 is the most stable winner from 6144 onward, staying around 6.16-6.18 GFLOP/s.
- At 8192, Block 512 is about 54% faster than OnMultLine (6.16 vs 4.01 GFLOP/s).

**Best large-matrix choice:
OnMultBlock with block 512**

OpenMP mapping with 4 threads

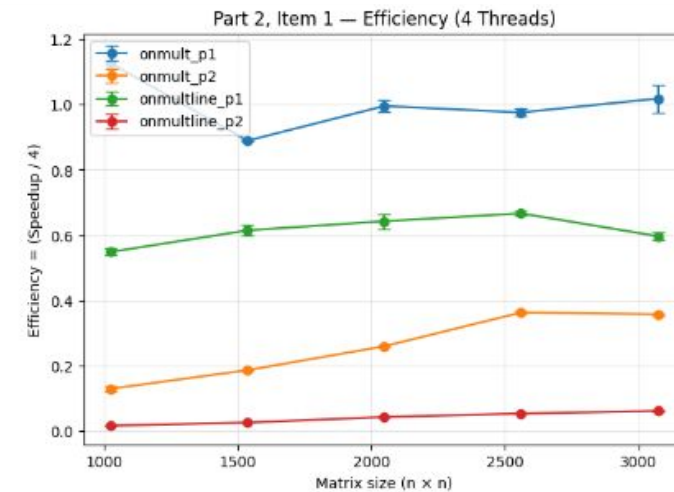
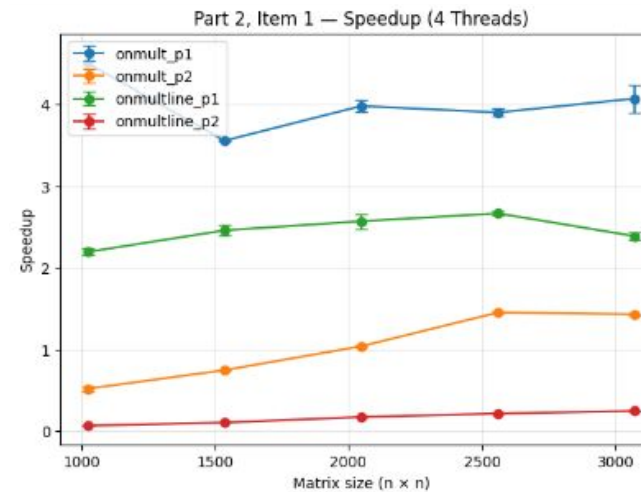
Different ways of parallelizing the loops lead to very different speedup and absolute performance.



Strategy 1: parallelizes the outer loop, so each thread computes independent rows/elements of the result matrix.

Strategy 2: parallelizes the inner accumulation loop, so multiple threads collaborate on the same result element.

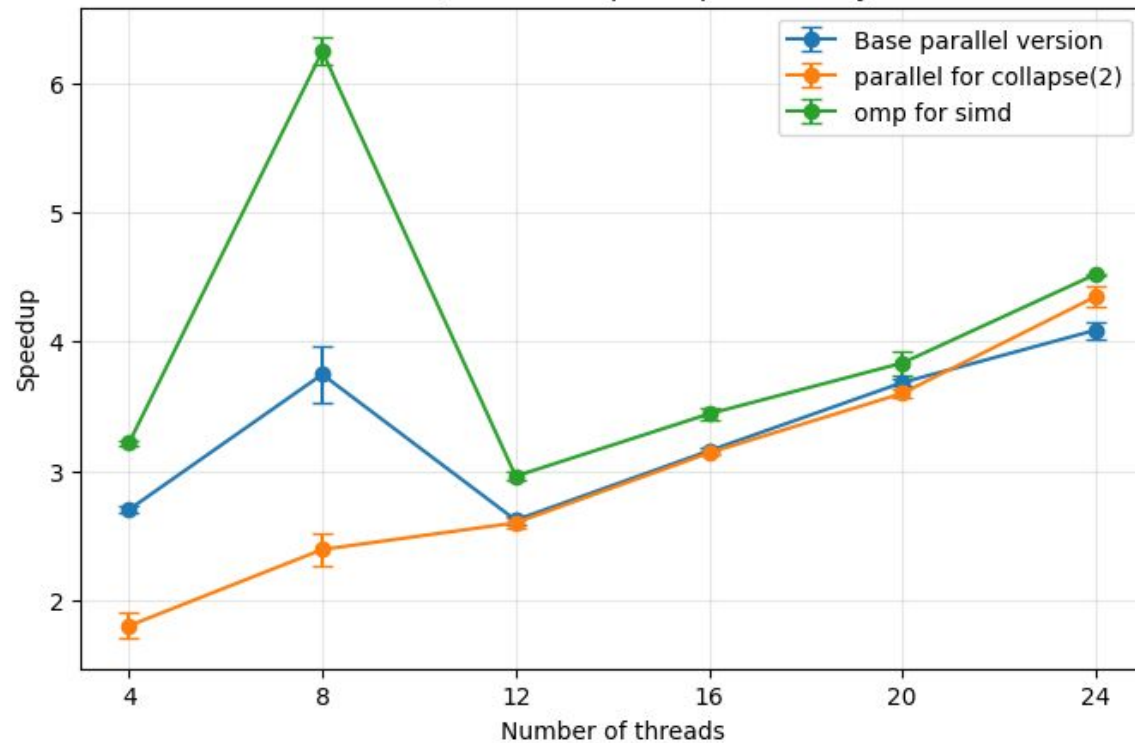
- OnMultLine - Strategy 1 gives the best absolute throughput, peaking around 13.4 GFLOP/s and staying above 10.4 GFLOP/s.
- OnMult - Strategy 1 reaches the highest speedup, close to 4x, but from a much slower single-core baseline.
- The choice of mapping strategy matters almost as much as the number of threads.



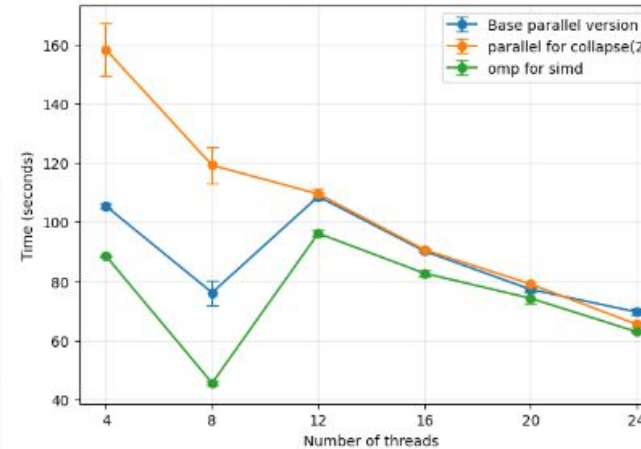
Scaling at 8192 x 8192 with advanced OpenMP pragmas

We compared the base parallel version with collapse(2) and omp for simd from 4 to 24 threads.

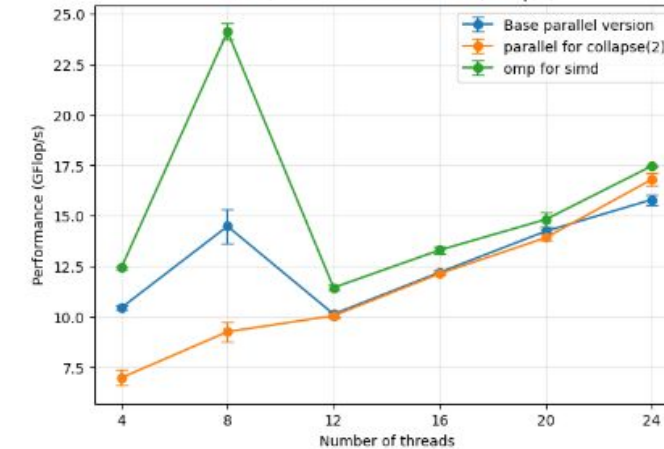
Part 2, Item 2 — Speedup Scalability



Part 2, Item 2 — Execution Time



Part 2, Item 2 — Performance (GFlop/s)



- omp for simd is the best variant overall and reaches the best point of the whole study: about 24 GFLOP/s at 8 threads.
- At 24 threads, omp for simd still leads with about 17.5 GFLOP/s, while collapse(2) is the second-best option.
- Scaling is non-linear, which suggests limits from memory bandwidth and scheduling overhead.

Key takeaways and final recommendation

The results consistently show that memory locality dominates performance in matrix multiplication.

1. Baseline OnMult is memory-bound

It stays below 0.85 GFLOP/s and degrades quickly as matrix size increases.

2. OnMultLine proves locality is the main lever

Just reordering the loops lifts performance by about 6x to 11x in the tested range.

3. Blocking is best for large matrices

For 6144 to 10240, Block 512 is the most stable and fastest single-core configuration.

4. OpenMP helps, but the strategy matters

The best overall result comes from OnMultLine with omp for simd, peaking around 24 GFLOP/s.

Recommended choices: OnMultLine for small/medium matrices, OnMultBlock (Block 512) for large matrices, and omp for simd for the best multicore performance.